

Lumen3D — Guide de création de plugins

Outils de la toolbar, contrôles par canal, et shaders

Plateforme Web IRIBHM / Lumen3D

Version plateforme 1.1.0

Table des matières

1	Vue d'ensemble	2
1.1	Les trois emplacements (« placements »)	2
1.2	Comment la détection fonctionne (résolution hybride)	2
2	Anatomie d'un plugin	2
2.1	Cycle de vie	2
3	Référence : plugin.json	3
4	Le ViewerContext (ctx)	3
5	Type 1 — Outils de la toolbar	4
5.1	Où le bouton apparaît : le champ <code>group</code>	4
5.2	Le comportement du bouton : le champ <code>subtype</code>	4
5.3	Exemple A — une action simple	4
5.4	Exemple B — une bascule avec menu latéral auto-créé	5
5.5	Exemple C — un outil ToolManager exclusif	6
5.6	Visibilité conditionnelle (<code>requires</code>)	6
5.7	Internationalisation	7
6	Type 2 — Contrôles par canal	7
6.1	Contrat	7
6.2	Exemple — un curseur d'opacité par canal	7
7	Type 3 — Shaders (modes de rendu)	8
7.1	Contrat et <code>plugin.json</code>	8
7.2	Nuance importante : réutiliser vs créer un mode de rendu	9
8	Déploiement & découverte	9
9	Bonnes pratiques & pièges	9
10	Aide-mémoire (checklist)	9

1 Vue d'ensemble

Depuis la version 1.1.0, le viewer (`viewer.html`) est **totalelement modulaire**. On ajoute une fonctionnalité en déposant un dossier dans `js/modules/`, et on la retire en supprimant ce dossier — *sans toucher au code de la plateforme*. La plateforme détecte les plugins présents au chargement et incorpore automatiquement leurs contrôles.

1.1 Les trois emplacements (« placements »)

Placement	Dossier	Où apparaît le plugin
tools	<code>js/modules/tools/</code>	Boutons de la barre d'outils en haut du viewer
channels	<code>js/modules/channels/</code>	Contrôles ajoutés sous chaque canal, dans le menu latéral
shaders	<code>js/modules/shaders/</code>	Entrées du menu déroulant « Render Mode » (mode de rendu 3D)

1.2 Comment la détection fonctionne (résolution hybride)

`PluginRegistry.discover()` résout la liste des plugins selon un ordre de repli robuste, de sorte que le viewer démarre toujours :

1. **Endpoint live** `GET /api/plugins` — servi par `dev_server.py` (et `api/plugins.php` pour PHP). Scanne `js/modules/` en direct : déposer/retirer un dossier est pris en compte au rechargement, sans aucune étape manuelle.
2. **Manifeste statique** `js/modules/manifest.json` — pour les hôtes purement statiques (`fast_server.py`, `python -m http.server`). Régénéré automatiquement par l'endpoint, ou à la main avec `tools/gen_plugins_manifest.py`.
3. **Liste cœur embarquée** dans `plugin-registry.js` — plancher anti-crash si rien d'autre n'est joignable.

En pratique : si vous développez avec `dev_server.py` (recommandé), il n'y a *rien* à faire d'autre que créer le dossier. Voir la section 8 pour les hôtes statiques.

2 Anatomie d'un plugin

Tout plugin, quel que soit son type, est un dossier contenant exactement deux fichiers :

```
js/modules/<placement>/<mon-plugin>/
|-- plugin.json   # metadonnees (id, nom, placement, icone, ...)
|-- index.js      # implementation : appelle PluginRegistry.implement(...)
```

Règles :

- Le nom du dossier doit correspondre au champ `id` de `plugin.json` et respecter `[A-Za-z0-9_][A-Za-z0-9._-]` (pas d'espaces, pas de point initial).
- Le `placement` déclaré dans `plugin.json` doit correspondre au dossier parent (`tools/channels/shaders`), sinon le plugin est rejeté.
- `index.js` ne fait *que* appeler `PluginRegistry.implement(id, impl)` ; le corps n'est pas exécuté avant que la plateforme n'appelle `init(ctx)`.

2.1 Cycle de vie

1. `discover()` puis `loadModules()` : la plateforme lit `plugin.json` (métadonnées enregistrées) puis injecte `index.js` (qui appelle `implement`).

2. `buildToolBarButtons()` : génère les boutons des outils depuis les métadonnées (placement `tools` uniquement).
3. `initAll(ctx)` : appelle `init(ctx)` de chaque plugin, en lui passant le *ViewerContext*. La valeur retournée devient `this` pour les appels `activate()` ultérieurs.
4. `bindToolBarButtons()` : connecte le clic des boutons générés (`data-plugin-id`) à `activate()`.
5. À l'extinction / changement de dataset : `disposeAll()` appelle `dispose()` de chaque plugin.

Invariant à respecter : tout le travail lourd doit rester hors du thread principal (workers), et toute texture / géométrie Three.js allouée par un plugin doit être libérée dans `dispose()`.

3 Référence : `plugin.json`

Champ	Placement	Rôle
<code>id</code>	tous	Identifiant unique = nom du dossier.
<code>name</code>	tous	Nom lisible (menu déroulant des shaders, repli de tooltip).
<code>placement</code>	tous	<code>tools</code> / <code>channels</code> / <code>shaders</code> .
<code>version</code>	tous	SemVer du plugin (informatif).
<code>creator</code>	tous	Auteur (informatif).
<code>description</code>	tous	Description courte (informatif).
<code>order</code>	tous	Entier de tri (croissant) dans son groupe / sa liste.
<code>group</code>	<code>tools</code>	Cluster de la toolbar : <code>tools</code> , <code>export</code> , <code>visuals</code> , <code>layouts</code> .
<code>subtype</code>	<code>tools</code>	<code>action</code> / <code>toggle</code> / <code>tool</code> (voir §5).
<code>icon</code>	<code>tools</code>	Nom d'icône Lucide (ex. <code>camera</code> , <code>ruler</code>).
<code>i18nTitle</code>	<code>tools</code>	Clé i18n du tooltip (repli sur <code>name</code>).
<code>i18nAria</code>	<code>tools</code>	Clé i18n de l' <code>aria-label</code> (repli sur <code>i18nTitle</code>).
<code>buttonId</code>	<code>tools</code>	<code>id</code> HTML du bouton généré (optionnel).
<code>tool</code>	<code>tools</code>	(subtype <code>tool</code>) nom d'outil <code>ToolManager</code> (valeur <code>data-tool</code>).
<code>shortcut</code>	<code>tools</code>	(subtype <code>tool</code>) raccourci clavier (ex. <code>m</code>).
<code>requires</code>	<code>tools</code>	Tableau de kind de <code>volumeSources</code> requis pour afficher le bouton.
<code>renderModeValue</code>	<code>shaders</code>	Entier passé à <code>VolumeViewer.setRenderMode()</code> (voir §7).
<code>default</code>	<code>shaders</code>	<code>true</code> pour le mode sélectionné par défaut.

Tous les champs au-delà des identifiants de base sont optionnels et ont un repli : les `plugin.json` existants restent valides.

4 Le `ViewerContext` (`ctx`)

`init(ctx)` reçoit une façade stable vers la plateforme. **N'accédez jamais aux singletons internes directement** (`VolumeViewer`, etc.) : passez toujours par `ctx`, ce qui garde votre plugin découplé.

Sous-objet	Méthodes principales
ctx.dataset	getMeta(), getId(), getBasePath()
ctx.viewer	getRenderer(), getMaterial(), getScene(), getCamera(), setRenderMode(n), setClipRange(...), resetClipping(), setGridMode(m), setAxesVisible(v), setVolumeVisible(v), setView(v), setRotationLocked(v), resize(), setCutPlaneVisible(v), setMeasurements(m), onMeasurePoint(cb), onPlaneSpecChange(cb), getPhysicalCalibration()
ctx.slicer	init/setVisible/isVisible/setPlaneSpec/getPlaneSpec/getPreviewCanvas/updat
ctx.channels	getState(), setState(s, opts)
ctx.measurements	list/add/update/remove/clear/setAll(scope, ...)
ctx.ui	toast(msg), scheduleResize(), escapeHtml(s), createIcons(opts), openStudio(), perf()
ctx.iframe	isIframe(), panelIndex(), postMessage(data)
ctx.workspace	getState(), applyState(s)
ctx._state	État mutable partagé (z-stack, sync) — via getters/setters.

5 Type 1 — Outils de la toolbar

5.1 Où le bouton apparaît : le champ group

La barre d'outils est divisée en clusters. Le champ **group** place votre bouton dans l'un d'eux :

group	Cluster (libellé)
tools	« Tools » — outils interactifs (à côté de l'outil cœur <i>Navigate</i>)
export	« Export » — capture, sauvegarde/restauration d'espace de travail
visuals	« Visuals » — bascules d'affichage (grille, axes, volume...)
layouts	« Layouts » — modes/vues (présentation, decompose, z-stack...)

À l'intérieur d'un cluster, l'ordre suit **order** (croissant). Un **group** inconnu n'a pas de cluster : le bouton est ignoré (avec un avertissement console).

5.2 Le comportement du bouton : le champ subtype

- **action** — bouton « one-shot » (ex. capture d'écran). **activate()** exécute l'action.
- **toggle** — bouton on/off. **activate()** renvoie { **active**: bool }; la plateforme bascule automatiquement le style **btn-solid**/**btn-ghost** et, optionnellement, l'icône ({ **active**, **icon** }).
- **tool** — outil *exclusif* géré par **ToolManager** (comme *Navigate*). Génère un chip **data-tool** ; déclarez **tool** (nom de l'outil) et éventuellement **shortcut**.

5.3 Exemple A — une action simple

js/modules/tools/say-hello/plugin.json

```
{
  "id": "say-hello",
  "name": "Say Hello",
  "placement": "tools",
  "group": "visuals",
  "subtype": "action",
  "icon": "sparkles",
  "order": 99,
  "i18nTitle": "Say hello"
```

```
}
```

```
js/modules/tools/say-hello/index.js
```

```
/* Say Hello -- a minimal one-shot tool */
PluginRegistry.implement('say-hello', {
  init(ctx) { this._ctx = ctx; return this; },

  activate() {
    const name = this._ctx.dataset.getMeta()?.name || 'dataset';
    this._ctx.ui.toast('Hello from ${name}!');
  },

  dispose() {}
});
```

C'est tout : déposer ce dossier fait apparaître un bouton « étincelles » dans le cluster *Visuals*. Le supprimer le retire.

5.4 Exemple B — une bascule avec menu latéral auto-crée

C'est le cas le plus demandé : un bouton qui ouvre/ferme un panneau dans la barre latérale. **Pour rester totalement autonome, le plugin crée lui-même son panneau** (au lieu de dépendre de HTML statique). On l'injecte dans la barre latérale `#viewer-sidebar` et on bascule sa visibilité.

```
js/modules/tools/notes-panel/plugin.json
```

```
{
  "id": "notes-panel",
  "name": "Notes",
  "placement": "tools",
  "group": "layouts",
  "subtype": "toggle",
  "icon": "notebook-pen",
  "order": 50,
  "i18nTitle": "Open notes panel"
}
```

```
js/modules/tools/notes-panel/index.js
```

```
/* Notes -- a toggle tool that creates its own sidebar panel */
PluginRegistry.implement('notes-panel', {
  _open: false,

  init(ctx) {
    this._ctx = ctx;
    // Create the panel once, append it to the viewer sidebar.
    const sidebar = document.getElementById('viewer-sidebar');
    const section = document.createElement('div');
    section.className = 'panel-section';
    section.id = 'notes-panel-section';
    section.style.display = 'none'; // hidden until toggled
    section.innerHTML = `
      <div class="panel-title"><span>Notes</span></div>
      <textarea id="notes-area" rows="6" style="width:100%"
        placeholder="Free notes about this dataset..."></textarea>`;
    sidebar.appendChild(section);
    this._section = section;
    // Persist per dataset via localStorage (key scoped by id).
    const key = 'notes:' + ctx.dataset.getId();
    const area = section.querySelector('#notes-area');
```

```

    area.value = localStorage.getItem(key) || '';
    area.addEventListener('input', () => localStorage.setItem(key, area.value));
    return this;
  },

  // Toggle: return { active } so the registry styles the button.
  activate() {
    this._open = !this._open;
    this._section.style.display = this._open ? '' : 'none';
    return { active: this._open };
  },

  // Workspace persistence (optional): saved/restored across sessions.
  getState() { return { open: this._open }; },
  setState(s) {
    this._open = !!s && s.open;
    if (this._section) this._section.style.display = this._open ? '' : 'none';
  },

  dispose() { this._section?.remove(); }
});

```

Pourquoi auto-cr  er le DOM ? Les plugins historiques (z-stack, slice inspector...) s'appuient sur du HTML pr  -  crit dans `viewer.html` et s'y attachent par id. C'est fonctionnel mais *non* drop-in (il faudrait   diter `viewer.html`). Pour un nouveau plugin autonome, cr  ez votre DOM dans `init()` et lib  rez-le dans `dispose()`.

5.5 Exemple C — un outil ToolManager exclusif

Un subtype: "tool" devient l'outil actif exclusif (le curseur de la souris sur le canevas lui est d  di  , comme *Measure*). D  clarez tool (la valeur data-tool) et un shortcut clavier facultatif.

```

{
  "id": "my-picker", "name": "Picker", "placement": "tools",
  "group": "tools", "subtype": "tool", "tool": "pick",
  "shortcut": "p", "icon": "crosshair", "order": 30,
  "i18nTitle": "Pick a point"
}

```

```

PluginRegistry.implement('my-picker', {
  init(ctx) {
    this._ctx = ctx;
    // React to 3D picks; behaviour depends on the active ToolManager tool.
    ctx.viewer.onMeasurePoint(pt => {
      if (typeof ToolManager !== 'undefined' && ToolManager.current() !== 'pick') return;
      ctx.ui.toast('Picked: ' + JSON.stringify(pt.normalized));
    });
    return this;
  },
  dispose() {}
});

```

Le raccourci p (d  clar   dans `plugin.json`) active l'outil automatiquement : ToolManager lit les raccourcis des plugins de subtype: "tool" au d  marrage.

5.6 Visibilit   conditionnelle (requires)

Pour n'afficher un bouton que si le dataset offre une source donn  e, d  clarez **requires** (liste de kind de volumeSources). Exemple du plugin deepzoom :

```
{ "id": "deepzoom-2d", "placement": "tools", "group": "layouts",
  "subtype": "toggle", "requires": ["deepzoom2d"], "...": "..." }
```

5.7 Internationalisation

Mettez la *clé* `i18n` dans `i18nTitle/i18nAria` et ajoutez sa traduction dans `lang/en.json`, `lang/fr.json`, `lang/es.json`. La plateforme retraduit les boutons générés automatiquement. Si la clé n'existe pas, la clé brute (ou `name`) est affichée.

6 Type 2 — Contrôles par canal

Un plugin `channels` ajoute des contrôles *sous chaque canal* du menu latéral (comme l'histogramme ou le filtre gaussien). Le panneau de canaux appelle votre plugin pour *chaque* canal : vous fournissez le HTML, puis vous le câblez.

6.1 Contrat

Méthode	Rôle
<code>init(ctx)</code>	Initialisation (comme les autres types).
<code>getChannelUI(channel)</code>	Retourne une <i>chaîne HTML</i> insérée dans la ligne du canal. Suffixez vos <code>id</code> par <code>channel.idx</code> pour rester unique.
<code>bindChannelUI(idx, channel, container, callbacks)</code>	Câble les événements. <code>callbacks = { getState, onStateChange, getHistograms }</code> .
<code>syncUI(idx, channel, container, getHistograms)</code>	(Re)synchronise l'UI quand l'état du canal change (appelé après chaque modification).
<code>dispose()</code>	Nettoyage.

L'objet `channel` expose notamment : `idx`, `min`, `max`, `midtone`, `gamma`, `color`, `enabled`, `opacity`, `name`. Les `callbacks` : `getState(idx)` lit l'état courant, `onStateChange(idx, state)` applique une modification (et redéclenche le rendu), `getHistograms()[idx]` donne `{ counts: [...], total }`.

6.2 Exemple — un curseur d'opacité par canal

`js/modules/channels/opacity-slider/plugin.json`

```
{
  "id": "opacity-slider",
  "name": "Opacity Slider",
  "placement": "channels",
  "subtype": "per-channel",
  "order": 20
}
```

`js/modules/channels/opacity-slider/index.js`

```
PluginRegistry.implement('opacity-slider', {
  init(ctx) { this._ctx = ctx; return this; },
```

```

// 1. HTML injected under each channel (ids suffixed by idx).
getChannelUI(channel) {
  return `
    <div class="channel-advanced-row">
      <label class="text-xs text-muted">Opacity
      <input type="range" id="op-${channel.idx}" min="5" max="100"
        value="${Math.round((channel.opacity ?? 1) * 100)}">
      </label>
    </div>`;
},

// 2. Wire the control to the channel state.
bindChannelUI(idx, channel, container, { getState, onStateChange }) {
  const slider = container.querySelector(`#op-${idx}`);
  if (!slider) return;
  slider.addEventListener('input', () => {
    const state = getState(idx);
    state.opacity = Number(slider.value) / 100;
    onStateChange(idx, state); // applies + re-renders
  });
},

// 3. Keep the UI in sync when the state changes elsewhere.
syncUI(idx, channel, container) {
  const slider = container.querySelector(`#op-${idx}`);
  if (slider) slider.value = Math.round((channel.opacity ?? 1) * 100);
},

dispose() {}
});

```

Déposer ce dossier ajoute un curseur d'opacité sous chaque canal; le supprimer le retire de tous les canaux.

7 Type 3 — Shaders (modes de rendu)

Un plugin `shaders` ajoute une entrée au menu déroulant « Render Mode ». `activate()` bascule le mode de rendu via `VolumeViewer.setRenderMode(n)`.

7.1 Contrat et `plugin.json`

`js/modules/shaders/my-shader/plugin.json`

```

{
  "id": "my-shader",
  "name": "My Shader",
  "placement": "shaders",
  "renderModeValue": 1,
  "default": false,
  "order": 30
}

```

```

PluginRegistry.implement('my-shader', {
  init(ctx) { this._ctx = ctx; return this; },
  activate() { this._ctx.viewer.setRenderMode(1); }, // 0 = DVR, 1 = Emission
  dispose() {}
});

```

7.2 Nuance importante : réutiliser vs créer un mode de rendu

Le `renderModeValue` est un *entier* que le shader GLSL du ray-marcher sait interpréter. Deux cas :

- **Réutiliser un mode existant** (0 = Direct Volume Rendering, 1 = émission/fluorescence) : *pur drop-in*. Votre plugin propose simplement une autre entrée de menu qui sélectionne un rendu déjà implémenté — aucune autre modification.
- **Créer un nouvel algorithme de rendu** : il faut, *en plus* du plugin, ajouter la branche GLSL correspondante dans `js/viewers/volume-viewer.js` (le `switch` sur `renderMode` dans le fragment shader). Le plugin reste l'entrée de menu + l'activation ; le moteur doit connaître le nouveau code de rendu. C'est la seule partie qui n'est pas « dossier seul ».

8 Déploiement & découverte

- **Développement** (`dev_server.py`) : rien à faire. Le dossier est détecté au rechargement, et `js/modules/manifest.json` est réécrit à chaque appel de l'endpoint.
- **Hôte statique** (`fast_server.py`, `python -m http.server`) ou **PHP** sans avoir lancé le dev server : régénérez le manifeste une fois :

```
python tools/gen_plugins_manifest.py
```

Committez `js/modules/manifest.json` pour les déploiements statiques.

- **Vérifier** la découverte : ouvrez `/api/plugins` (doit lister votre plugin) ; la console du viewer affiche `[PluginRegistry] Discovered N plugins via ....`

9 Bonnes pratiques & pièges

- **Toujours libérer** dans `dispose()` : DOM créé, textures / géométries Three.js, écouteurs globaux, timers. (Règle d'hygiène GPU.)
- **Pas de travail lourd sur le thread principal** : décodage, flou, parsing → Web Worker (voir les workers existants).
- **Passez par `ctx`**, pas par les singletons internes : votre plugin reste découplé et survit aux refontes.
- **Idempotence** : la page *Compare* monte N iframes du viewer ; votre `init()` s'exécute une fois par panneau. Évitez les effets de bord globaux non réentrants.
- **N'éditez pas `viewer.html`** pour un nouveau plugin : créez votre DOM dans `init()`. Le HTML statique est réservé aux plugins historiques.
- **Tri** : utilisez `order` pour positionner votre bouton / entrée ; ne dépendez pas de l'ordre de chargement.
- **Robustesse** : un `plugin.json` invalide est ignoré (le reste continue de charger) — mais vérifiez votre JSON.

10 Aide-mémoire (checklist)

1. Créer `js/modules/<placement>/<id>/`.
2. Écrire `plugin.json` (`id` = nom du dossier ; `placement` = dossier parent).
3. Écrire `index.js` : `PluginRegistry.implement(id, { ... })`.
4. Tools : choisir `group` (cluster) + `subtype` ; ajouter `icon` et `i18nTitle`.
5. Channels : implémenter `getChannelUI` / `bindChannelUI` / `syncUI`.

6. Shaders : `activate()` → `setRenderMode(n)` ; ajouter la branche GLSL si nouveau mode.
7. Recharger le viewer (dev server) ou régénérer le manifeste (statique).
8. Vérifier dans la console et tester le retrait du dossier.

Référence code : `js/core/plugin-registry.js` (découverte, génération toolbar, cycle de vie) ; `js/components/channel-p` (intégration channels) ; `js/pages/viewer.js` (construction du ViewerContext). Tests : `tests/js/test_plugin_autonomy`
`tests/test_dev_server_plugins.py`.